

Practice Questions

Q1. Take a different CFG (for a simple fragment of English or arithmetic expressions). Convert it to CNF and implement CYK to check membership for several sentences?

Ans->

- **Program:-**

```
from collections import defaultdict

class CNFGrammar:

    def __init__(self):
        self productions = defaultdict(list)
        self rhs_to_lhs = defaultdict(list)

    def add_rule(self, lhs, rhs):
        """Add a production rule to the grammar."""
        self productions[lhs].append(rhs)
        self rhs_to_lhs[rhs].append(lhs)

    def get_lhs_for_rhs(self, rhs):
        """Get all non-terminals that can produce the given RHS."""
        return self rhs_to_lhs.get(rhs, [])

    def cyk_parse(grammar, tokens, start_symbol="S"):
        """
        CYK algorithm implementation for parsing with CNF grammar.
        Returns (parse_table, back_pointers, is_accepted)
        """
        n = len(tokens)

        # Initialize tables
        table = [[set() for _ in range(n + 1)] for _ in range(n)]
```

```
back = {}

# Base case: handle terminals (length 1 substrings)
for i, token in enumerate(tokens):

    back[(i, 1)] = defaultdict(list)

    for nt in grammar.get_lhs_for_rhs(token):

        table[i][1].add(nt)

        back[(i, 1)][nt].append(('terminal', token))

# Fill the table for substrings of length 2 to n
for length in range(2, n + 1):

    for i in range(n - length + 1):

        back[(i, length)] = defaultdict(list)

        for k in range(1, length):

            # For each possible split position
            for A in table[i][k]:

                for B in table[i + k][length - k]:

                    # Check if there's a production rule that produces AB
                    for nt in grammar.get_lhs_for_rhs((A, B)):

                        if nt not in back[(i, length)]:

                            back[(i, length)].add(nt)

                            back[(i, length)][nt].append(('binary', k, A, B))

# Check if the start symbol is in the top-right cell
is_accepted = start_symbol in table[0][n]

return table, back, is_accepted

def build_trees(back, tokens, i, length, symbol):
```

```

"""Recursively build all possible parse trees."""
if (i, length) not in back or symbol not in back[(i, length)]:
    return []
trees = []
for production in back[(i, length)][symbol]:
    if production[0] == 'terminal':
        # Terminal production
        trees.append((symbol, tokens[i]))
    else:
        # Binary production
        _, k, A, B = production
        left_trees = build_trees(back, tokens, i, k, A)
        right_trees = build_trees(back, tokens, i + k, length - k, B)
        # Combine all possible left and right subtrees
        for l in left_trees:
            for r in right_trees:
                trees.append((symbol, l, r))
return trees

def tree_to_string(tree):
    """Convert parse tree to a string representation."""
    if len(tree) == 2: # Terminal
        return f"({tree[0]})"
    else: # Non-terminal
        return f"({tree[0]} {tree_to_string(tree[1])} {tree_to_string(tree[2])})"

# Define the CNF grammar for arithmetic expressions

```

```
grammar = CNFGrammar()

# Terminals
grammar.add_rule('N', '0')
grammar.add_rule('N', '1')
grammar.add_rule('N', '2')
grammar.add_rule('N', '3')
grammar.add_rule('N', '4')
grammar.add_rule('N', '5')
grammar.add_rule('N', '6')
grammar.add_rule('N', '7')
grammar.add_rule('N', '8')
grammar.add_rule('N', '9')

# Non-terminals for handling numbers with multiple digits
grammar.add_rule('D', 'N')
grammar.add_rule('D', ('N', 'D'))

# Expression rules
grammar.add_rule('E', 'N') #  $E \rightarrow N$ 
grammar.add_rule('E', 'D') #  $E \rightarrow D$ 

# Binary operations
grammar.add_rule('E', ('E', 'P')) #  $E \rightarrow E P$ 

# Plus operation
grammar.add_rule('P', ('+', 'E')) #  $P \rightarrow + E$ 

# Minus operation
grammar.add_rule('P', ('-', 'E')) #  $P \rightarrow - E$ 
```

```
# Multiplication and division (higher precedence)
```

```
grammar.add_rule('E', ('E', 'T')) #  $E \rightarrow E T$ 
```

```
# Times operation
```

```
grammar.add_rule('T', ('*', 'F')) #  $T \rightarrow * F$ 
```

```
# Division operation
```

```
grammar.add_rule('T', ('/', 'F')) #  $T \rightarrow / F$ 
```

```
# Factor (for handling parentheses)
```

```
grammar.add_rule('F', 'E') #  $F \rightarrow E$ 
```

```
grammar.add_rule('F', ('(', 'E')) #  $F \rightarrow ( E$ 
```

```
grammar.add_rule('F', ('E', ')')) #  $F \rightarrow E )$ 
```

```
# Start symbol
```

```
grammar.add_rule('S', 'E')
```

```
# Test the parser
```

```
test_expressions = [
```

```
    "1+2*3",
```

```
    "(1+2)*3",
```

```
    "10-5/2",
```

```
    "1+2+3+4",
```

```
    "2*(3+4)-5/1",
```

```
    "1++2", # Should be invalid
```

```
    "3*" # Should be invalid
```

```
]
```

```
for expr in test_expressions:
```

```
    tokens = list(expr)
```

```
table, back, accepted = cyk_parse(grammar, tokens)

print(f"\nExpression: {expr}")

if accepted:

    print("Status: Accepted")

    # Get all parse trees

    parse_trees = build_trees(back, tokens, 0, len(tokens), 'S')

    if parse_trees:

        print("Parse tree:")

        print(tree_to_string(parse_trees[0])) # Print first parse tree

    else:

        print("Status: Rejected")
```

- **Output:-**

```
PS D:\NLP\EXP7> python -u "d:\NLP\EXP7\tempCodeRunnerFile.py"

Expression: 1+2*3
Status: Rejected

Expression: (1+2)*3
Status: Rejected

Expression: 10-5/2
Status: Rejected

Expression: 1+2+3+4
Status: Rejected

Expression: 2*(3+4)-5/1
Status: Rejected

Expression: 1++2
Status: Rejected

Expression: 3*
Status: Rejected
```

Q2. Modify the program to compute the number of distinct parses for ambiguous grammars (use counts as in the code). Test with an ambiguous grammar (e.g., arithmetic grammar without precedence)?

Ans->

- **Program:-**

```
from collections import defaultdict
from functools import lru_cache

class CNFGrammar:
    def __init__(self):
        self productions = defaultdict(list)
        self rhs_to_lhs = defaultdict(list)
        self._count_cache = {}

    def add_rule(self, lhs, rhs):
        """Add a production rule to the grammar."""
        self productions[lhs].append(rhs)
        self rhs_to_lhs[rhs].append(lhs)
        self._count_cache = {} # Clear cache when grammar changes

    def get_lhs_for_rhs(self, rhs):
        """Get all non-terminals that can produce the given RHS."""
        return self rhs_to_lhs.get(rhs, [])

    def cyk_parse_count(grammar, tokens, start_symbol="S"):
        """
        Modified CYK algorithm that counts the number of distinct parse trees.
        Returns (count, back_pointers)
        """
        n = len(tokens)
```

```
# Initialize count and back tables
count = [[defaultdict(int) for _ in range(n + 1)] for _ in range(n)]
back = {}

# Base case: handle terminals (length 1 substrings)
for i, token in enumerate(tokens):
    back[(i, 1)] = defaultdict(list)
    for nt in grammar.get_lhs_for_rhs(token):
        count[i][1][nt] = 1
        back[(i, 1)][nt].append(('terminal', token))

# Fill the count table for substrings of length 2 to n
for length in range(2, n + 1):
    for i in range(n - length + 1):
        back[(i, length)] = defaultdict(list)
        for k in range(1, length):
            # For each possible split position
            for A in count[i][k]:
                for B in count[i + k][length - k]:
                    # For each production that can produce AB
                    for nt in grammar.get_lhs_for_rhs((A, B)):
                        # Add the product of counts from left and right
                        count[i][length][nt] += count[i][k][A] * count[i + k][length - k][B]
                        back[(i, length)][nt].append(('binary', k, A, B))

# Return the count for the start symbol spanning the entire input
return count[0][n].get(start_symbol, 0), back

def build_parse_tree(back, tokens, i, length, symbol, tree_id=0):
```



```
''''''
```

Build a specific parse tree based on tree_id.

This is a helper function for tree visualization.

```
''''''
```

```
if (i, length) not in back or symbol not in back[(i, length)]:
```

```
    return None
```

```
if back[(i, length)][symbol][0][0] == 'terminal':
```

```
    return (symbol, tokens[i])
```

```
else:
```

```
    # For binary productions, we need to select which production to use
```

```
    # based on tree_id
```

```
    productions = back[(i, length)][symbol]
```

```
    prod_idx = tree_id
```

```
    for prod in productions:
```

```
        if prod[0] == 'binary':
```

```
            _, k, A, B = prod
```

```
            # For each possible split of counts between left and right
```

```
            left_count = len([p for p in back[(i, k)][A] if p[0] in ['terminal', 'binary']])
```

```
            if prod_idx < left_count:
```

```
                left_tree = build_parse_tree(back, tokens, i, k, A, prod_idx)
```

```
                right_tree = build_parse_tree(back, tokens, i + k, length - k, B, 0)
```

```
                if left_tree and right_tree:
```

```
                    return (symbol, left_tree, right_tree)
```

```
            prod_idx -= left_count
```

```
    return None
```

```
def tree_to_string(tree):
    """Convert parse tree to a string representation."""
    if tree is None:
        return "None"
    if len(tree) == 2: # Terminal
        return f"({tree[0]} {tree[1]})"
    else: # Non-terminal
        return f"({tree[0]} {tree_to_string(tree[1])} {tree_to_string(tree[2])})"

# Define an ambiguous CNF grammar for arithmetic expressions (no operator precedence)
grammar = CNFGrammar()

# Terminals
for d in '0123456789':
    grammar.add_rule('N', d)

# Expression rules - all operations at the same level
grammar.add_rule('E', 'N') # E → N
grammar.add_rule('E', ('E', 'E+')) # E → E E+
grammar.add_rule('E', ('E', 'E*')) # E → E E*
grammar.add_rule('E+', ('+', 'E')) # E+ → + E
grammar.add_rule('E*', ('*', 'E')) # E* → * E

# Add rules for parentheses
grammar.add_rule('E', ('(', 'E')) # E → ( E
grammar.add_rule('E)', ('E', ')')) # E) → E )

# Start symbol
grammar.add_rule('S', 'E')

# Test the parser with ambiguous expressions
```

```
test_expressions = [  
    "1+2+3", # Ambiguous: could be (1+2)+3 or 1+(2+3)  
    "1*2+3", # Ambiguous: could be (1*2)+3 or 1*(2+3)  
    "1+2*3", # Ambiguous: could be (1+2)*3 or 1+(2*3)  
    "1+2+3+4", # More complex ambiguity  
    "1*2*3+4", # Multiple operations with different precedences  
]  
  
for expr in test_expressions:  
    tokens = list(expr)  
    count, back = cyk_parse_count(grammar, tokens)  
    print(f"\nExpression: {expr}")  
    print(f"Number of distinct parses: {count}")  
    if count > 0:  
        # Show the first few parse trees (up to 3)  
        max_trees_to_show = min(3, count)  
        print(f"Showing {max_trees_to_show} of {count} possible parse trees:")  
        for i in range(max_trees_to_show):  
            tree = build_parse_tree(back, tokens, 0, len(tokens), 'S', i)  
            if tree:  
                print(f"Tree {i+1}: {tree_to_string(tree)}")  
    else:  
        print("No valid parses found.")
```

- **Output:-**

```
PS D:\NLP\EXP7> python -u "d:\NLP\EXP7\tempCodeRunnerFile.py"

Expression: 1+2+3
Number of distinct parses: 0
No valid parses found.

Expression: 1*2+3
Number of distinct parses: 0
No valid parses found.

Expression: 1+2*3
Number of distinct parses: 0
No valid parses found.

Expression: 1+2+3+4
Number of distinct parses: 0
No valid parses found.

Expression: 1*2*3+4
Number of distinct parses: 0
No valid parses found.
```

Q3. Implement a simple Chart Parser (Earley algorithm) and compare: which sentences each parser accepts, and performance differences on the same grammar?

Ans->

- **Program:-**

```
import time

from collections import defaultdict, namedtuple
from functools import lru_cache

class Rule:

    def __init__(self, lhs, rhs, dot_pos=0, start_pos=0, end_pos=0):
        self.lhs = lhs
        self.rhs = tuple(rhs)
        self.dot_pos = dot_pos
        self.start_pos = start_pos
```

```
        self.end_pos = end_pos
    def __eq__(self, other):
        if not isinstance(other, Rule):
            return False
        return (self.lhs == other.lhs and
                self.rhs == other.rhs and
                self.dot_pos == other.dot_pos and
                self.start_pos == other.start_pos and
                self.end_pos == other.end_pos)
    def __hash__(self):
        return hash((self.lhs, self.rhs, self.dot_pos, self.start_pos, self.end_pos))
    def __repr__(self):
        rhs = list(self.rhs[:])
        rhs.insert(self.dot_pos, '•')
        return f"{self.lhs} → {' '.join(rhs)} [{self.start_pos}, {self.end_pos}]"
    def next_symbol(self):
        if self.dot_pos < len(self.rhs):
            return self.rhs[self.dot_pos]
        return None
    def is_complete(self):
        return self.dot_pos == len(self.rhs)
class CYKParser:
    def __init__(self, grammar):
        self.grammar = grammar
        self.table = None
```

```
def parse(self, tokens):
    n = len(tokens)
    # Initialize table
    self.table = [[set() for _ in range(n+1)] for _ in range(n)]

    # Base case: fill in terminals
    for i in range(n):
        for rule in self.grammar:
            if rule.rhs == (tokens[i],) and rule.dot_pos == 0:
                self.table[i][i+1].add(rule.lhs)

    # Fill the table
    for length in range(2, n+1):
        for i in range(n - length + 1):
            j = i + length
            for k in range(i+1, j):
                for rule in self.grammar:
                    if len(rule.rhs) == 2 and rule.dot_pos == 0:
                        A, B = rule.rhs
                        if A in self.table[i][k] and B in self.table[k][j]:
                            self.table[i][j].add(rule.lhs)

    # Check if start symbol is in the top-right cell
    return 'S' in self.table[0][n]

class EarleyParser:
    def __init__(self, grammar):
        self.grammar = grammar
```

```
self.chart = None
self.tokens = None
def parse(self, tokens):
    self.tokens = list(tokens)
    n = len(self.tokens)
    self.chart = [set() for _ in range(n + 1)]
    # Add dummy start rule
    start_rule = Rule('S\\', ['S'], 0, 0, 0)
    self.chart[0].add(start_rule)
    # Get all non-terminals from grammar
    non_terminals = {rule.lhs for rule in self.grammar}
    for i in range(n + 1):
        j = 0
        while j < len(self.chart[i]):
            rule = list(self.chart[i])[j]
            if not rule.is_complete():
                next_sym = rule.next_symbol()
                if next_sym in non_terminals: # Non-terminal
                    self.predict(rule, i)
                elif i < n and next_sym == self.tokens[i]: # Terminal
                    self.scan(rule, i)
            else:
                self.complete(rule, i)
            j += 1
    # Check if we've successfully parsed the entire input
```

```
final_rule = Rule('S', ['S'], 1, 0, n)
return final_rule in self.chart[n]

def predict(self, rule, pos):
    next_sym = rule.next_symbol()
    for prod in self.grammar:
        if prod.lhs == next_sym and prod.dot_pos == 0:
            new_rule = Rule(
                prod.lhs,
                prod.rhs,
                0, # dot at start
                pos,
                pos # start and end at current position
            )
            if new_rule not in self.chart[pos]:
                self.chart[pos].add(new_rule)

def scan(self, rule, pos):
    next_sym = rule.next_symbol()
    if pos < len(self.tokens) and next_sym == self.tokens[pos]:
        new_rule = Rule(
            rule.lhs,
            rule.rhs,
            rule.dot_pos + 1,
            rule.start_pos,
            pos + 1
        )
```



```
        if new_rule not in self.chart[pos + 1]:
            self.chart[pos + 1].add(new_rule)
def complete(self, rule, pos):
    for r in self.chart[rule.start_pos]:
        if not r.is_complete() and r.next_symbol() == rule.lhs:
            new_rule = Rule(
                r.lhs,
                r.rhs,
                r.dot_pos + 1,
                r.start_pos,
                pos
            )
            if new_rule not in self.chart[pos]:
                self.chart[pos].add(new_rule)
def test_parsers(grammar, test_cases):
    cyk_parser = CYKParser(grammar)
    earley_parser = EarleyParser(grammar)
    print(f'{"Test Case":<30} {"CYK":<10} {"Earley":<10} {"Time (CYK)":<12} {"Time (Earley)":<12}')
    print("-" * 80)
    for test in test_cases:
        # Test CYK
        start = time.time()
        cyk_result = cyk_parser.parse(test)
        cyk_time = (time.time() - start) * 1000 # in milliseconds
        # Test Earley
```

```
start = time.time()
earley_result = earley_parser.parse(test)
earley_time = (time.time() - start) * 1000 # in milliseconds
print(f"{' '.join(test):<30} {str(cyk_result):<10} {str(earley_result):<10} "
      f"{cyk_time:>8.3f}ms    {earley_time:>8.3f}ms")

grammar = [
    # Start rules
    Rule('S', ['E']),

    # Expression rules
    Rule('E', ['E', '+', 'T']),
    Rule('E', ['E', '-', 'T']),
    Rule('E', ['T']),

    # Term rules
    Rule('T', ['T', '*', 'F']),
    Rule('T', ['T', '/', 'F']),
    Rule('T', ['F']),

    # Factor rules
    Rule('F', ['N']),
    Rule('F', ['(', 'E', ')']),

    # Number rules (simplified)
    Rule('N', ['0']), Rule('N', ['1']), Rule('N', ['2']), Rule('N', ['3']),
    Rule('N', ['4']), Rule('N', ['5']), Rule('N', ['6']),
    Rule('N', ['7']), Rule('N', ['8']), Rule('N', ['9']),
]
```

Test cases

```
test_cases = [
    ['2', '+', '3', '*', '4'],          # Valid
    ['(', '2', '+', '3', ')', '*', '4'], # Valid with parentheses
    ['2', '+', '3', '*'],              # Invalid
    ['2', '+', '*', '3'],              # Invalid
    ['2', '3', '+', '4'],              # Invalid (missing operator)
    ['2', '+', '3', '+', '4', '*', '5'], # Longer valid expression
]
```

Run the comparison

```
print("=== Parser Comparison ===")
```

```
test_parsers(grammar, test_cases)
```

Performance test with a longer input

```
print("\n=== Performance Test with Longer Input ===")
```

```
long_expr = ['(' + ['1', '+'] * 3 + ['2'] + [')', '*', '3'] * 2
```

```
test_parsers(grammar, [long_expr])
```

• Output:-

```
PS D:\NLP\EXP7> python -u "d:\NLP\EXP7\tempCodeRunnerFile.py"
=== Parser Comparison ===
Test Case          CYK      Earley    Time (CYK)  Time (Earley)
-----
2 + 3 * 4          False    False     0.080ms     0.043ms
( 2 + 3 ) * 4      False    False     0.128ms     0.029ms
2 + 3 *            False    False     0.039ms     0.023ms
2 + * 3            False    False     0.036ms     0.022ms
2 3 + 4            False    False     0.040ms     0.024ms
2 + 3 + 4 * 5      False    False     0.147ms     0.029ms

=== Performance Test with Longer Input ===
Test Case          CYK      Earley    Time (CYK)  Time (Earley)
-----
( 1 + 1 + 1 + 2 ) * 3 ) * 3  False    False     0.783ms     0.036ms
```